

Q1.1

Orderings:

1. 1122
2. 1212
3. 1221
4. 2121
5. 2112
6. 2211

Q1.2

Orderings:

1. 1122
2. 2211

Q1.3

```
class mysem {
public:
    mysem(uint32_t init_value) : _counter(init_value){};
    void acquire() {
        bool swapped = false;
        while (not swapped) {
            auto old = _counter.load();
            if (old > 0) {
                swapped = _counter.compare_exchange_strong(old, old - 1);
            }
        }
        _counter--;
    };
    void release() {
        _counter++;
    };

private:
    std::atomic<uint32_t> _counter;
};
```

Q1.4

```
class mysem_hybrid {
public:
    explicit mysem_hybrid(uint32_t init_value) : _counter(init_value){};
```

```

void acquire() {
    bool swapped = false;
    int i = 100;
    while (not swapped) {
        auto old = _counter.load();
        if (old > 0) {
            swapped = _counter.compare_exchange_strong(old, old - 1);
        }
        i--;
        if (i == 0) {
            auto *counter_ptr = reinterpret_cast<uint32_t*>(&_counter);
            syscall(SYS_futex, counter_ptr, FUTEX_WAIT, 0);
        }
    }
    _counter--;
};

void release() {
    _counter++;
    auto *counter_ptr = reinterpret_cast<uint32_t*>(&_counter);
    syscall(SYS_futex, counter_ptr, FUTEX_WAKE, 1);
};

```

```

private:
    std::atomic<uint32_t> _counter;
};

```

Q1.5

I could add an atomic integer field `_sleeping` to the semaphore class storing the number of sleeping (waiting) threads. Before each call to `FUTEX_WAIT`, increment `_sleeping`. After each call to `FUTEX_WAKE`, decrement `_sleeping`. In `mysem::release` call `FUTEX_WAKE` only if `_sleeping > 0`.

Q2.1

Total GET size: $1 + 512 + 1024 = 1537 \text{ B} = 12_{296} \text{ b}$

Total SET size: $1 + 512 + 1024 + 1 = 1538 \text{ B} = 12_{304} \text{ b}$

Throughput for GETs: $0.9 * 10 = 9 \text{ Gbps} = 9 * 10^9 \text{ bps}$

Throughput for SETs: $0.1 * 10 = 1 \text{ Gbps} = 10^9 \text{ bps}$

It's assumed that the number of operations per second needs to be an integer because otherwise some operations would have a very high latency between request and response and hence might simply fail due to a timeout on the server or on the client.

Number of GETs per second: $9 * 10^9 / 12_296 = \text{floor}(731_945.3) = 731_945$

Number of SETs per second: $10^9 / 12_304 = \text{floor}(81_274.4) = 81_274$

Total number of operations per second: $731_945 + 81_274 = 813_219$

Q2.2

GETs per second: $0.9 * 130_000 = 117_000$

SETs per second: $0.1 * 130_000 = 13_000$

GETs processed per core per second: $1 \text{ s} / (200 * 10^{-6}) \text{ s} = 5_000$

SETs processed per core per second: $1 \text{ s} / (500 * 10^{-6}) \text{ s} = 2_000$

Cores needed for GETs: $117_000 / 5_000 = 23.4$

Cores needed for SETs: $13_000 / 2_000 = 6.5$

It's assumed that a single core can process both GET and SET requests.

Total number of cores needed: $\text{ceiling}(23.4 + 6.5) = \text{ceiling}(29.9) = 30$

Q2.3

1st experiment:

1_000 requests per second can be processed by a single core in a sequential manner since $1_000 < 5_000$. So there's no overhead due to blocking other threads / cores. What's more, there are 10_000 buckets so it's unlikely that within any given 200 microsec period there are 2 simultaneous requests to the same bucket. So there's little probability of mutex contention and hence little / no locking overhead. The latency is thus the same as for the lock-free implementation.

2nd experiment:

100_000 requests per second cannot be processed on a single core in a sequential manner. So they are processed on multiple cores. What's more, it's likely that multiple requests access the same bucket simultaneously (in the same 200 microsec period). Locking the shared_mutex in reader mode involves incrementing an atomic integer that stores the number of reader threads holding the shared_mutex on the bucket. This operation involves mutual exclusion and is blocking. Some threads get blocked until they can increment the atomic integer. The shared_mutex implementation is slower than the lock-free one (200 microsec).

3rd experiment:

100_000 requests per second are processed in parallel. The expected latency for lock-free solution would be smaller: $(200 + 500) / 2 = 350$ microsec. The shared_mutex implementation suffers from the overhead of reader (GET) threads incrementing the atomic integer and the writer (SET) threads acquiring the writer lock. When a writer thread acquires the writer lock, it needs to wait until all reader threads in the bucket have completed their task. When a writer thread holds the writer lock, no other writer or reader thread in the bucket can proceed.